

Studentu pazymiu sistema

V2.0

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 include Directory Reference	9
5.2 src Directory Reference	9
6 Class Documentation	11
6.1 Studentas Class Reference	11
6.1.1 Detailed Description	12
6.1.2 Constructor & Destructor Documentation	13
6.1.2.1 Studentas() [1/4]	13
6.1.2.2 Studentas() [2/4]	13
6.1.2.3 ~Studentas()	13
6.1.2.4 Studentas() [3/4]	13
6.1.2.5 Studentas() [4/4]	14
6.1.3 Member Function Documentation	14
6.1.3.1 AddPaz()	14
6.1.3.2 ClearPaz()	14
6.1.3.3 egz()	14
6.1.3.4 med()	14
6.1.3.5 MedlrVidSkaciavimas()	15
6.1.3.6 operator=() [1/2]	15
6.1.3.7 operator=() [2/2]	15
6.1.3.8 paz()	15
6.1.3.9 ReadStudent()	16
6.1.3.10 SetEgz()	16
6.1.3.11 SetMed()	16
6.1.3.12 SetVid()	16
6.1.3.13 spausdinti()	16
6.1.3.14 vid()	17
6.2 Timer Class Reference	17
6.2.1 Detailed Description	17
6.2.2 Constructor & Destructor Documentation	17
6.2.2.1 Timer()	17

6.2.3 Member Function Documentation	18
6.2.3.1 elapsed()	18
6.2.3.2 reset()	18
6.3 Zmogus Class Reference	18
6.3.1 Detailed Description	19
6.3.2 Constructor & Destructor Documentation	19
6.3.2.1 Zmogus() [1/2]	19
6.3.2.2 Zmogus() [2/2]	19
6.3.2.3 ~Zmogus()	19
6.3.3 Member Function Documentation	19
6.3.3.1 pavarde()	19
6.3.3.2 SetPavarde()	20
6.3.3.3 SetVardas()	20
6.3.3.4 spausdinti()	20
6.3.3.5 vardas()	20
6.3.4 Member Data Documentation	20
6.3.4.1 _pavarde	20
6.3.4.2 _vardas	20
7 File Documentation	21
7.1 include/student.h File Reference	21
7.1.1 Detailed Description	21
7.1.2 Function Documentation	22
7.1.2.1 operator<<()	22
7.1.2.2 operator>>()	22
7.2 student.h	22
7.3 include/student_io_deque.h File Reference	23
7.3.1 Detailed Description	24
7.3.2 Function Documentation	24
7.3.2.1 benchmarkProcessingFile()	24
7.3.2.2 containsDigit()	25
7.3.2.3 duomenulrasymasFaile()	25
7.3.2.4 duomenulrasymasKonsole()	25
7.3.2.5 fileRead()	26
7.3.2.6 fileTest()	26
7.3.2.7 GenerateStudentsFile()	26
7.3.2.8 getSortChoice()	26
7.3.2.9 inputas()	27
7.3.2.10 outputas()	27
7.3.2.11 RuleOfFiveTestas()	27
7.3.2.12 sortByUser()	27
7.3.2.13 SplitStudentsStrategy1()	28

7.3.2.14 SplitStudentsStrategy2()	28
7.3.2.15 SplitStudentsStrategy3()	28
7.4 student_io_deque.h	29
7.5 include/student_io_list.h File Reference	29
7.5.1 Function Documentation	30
7.5.1.1 benchmarkProcessingFile()	30
7.5.1.2 containsDigit()	30
7.5.1.3 duomenulrasympasFaile()	30
7.5.1.4 duomenulrasympasKonsole()	31
7.5.1.5 fileRead()	31
7.5.1.6 fileTest()	31
7.5.1.7 GenerateStudentsFile()	31
7.5.1.8 getSortChoice()	31
7.5.1.9 inputas()	31
7.5.1.10 outputas()	32
7.5.1.11 sortByUser()	32
7.5.1.12 SplitStudentsStrategy1()	32
7.5.1.13 SplitStudentsStrategy2()	32
7.5.1.14 SplitStudentsStrategy3()	32
7.6 student_io_list.h	32
7.7 include/student_io_vector.h File Reference	33
7.7.1 Function Documentation	33
7.7.1.1 benchmarkProcessingFile()	33
7.7.1.2 containsDigit()	34
7.7.1.3 duomenulrasympasFaile()	34
7.7.1.4 duomenulrasympasKonsole()	34
7.7.1.5 fileRead()	34
7.7.1.6 fileTest()	34
7.7.1.7 GenerateStudentsFile()	35
7.7.1.8 getSortChoice()	35
7.7.1.9 inputas()	35
7.7.1.10 outputas()	35
7.7.1.11 sortByUser()	35
7.7.1.12 SplitStudentsStrategy1()	35
7.7.1.13 SplitStudentsStrategy2()	36
7.7.1.14 SplitStudentsStrategy3()	36
7.8 student_io_vector.h	36
7.9 include/timer.h File Reference	36
7.9.1 Detailed Description	37
7.10 timer.h	37
7.11 include/utlis.h File Reference	37
7.11.1 Detailed Description	38

7.11.2 Function Documentation	38
7.11.2.1 intInput()	38
7.11.2.2 randomVardasPavarde()	38
7.11.3 Variable Documentation	38
7.11.3.1 mot_pavardes	38
7.11.3.2 vardai	38
7.11.3.3 vyr_pavardes	39
7.12 utils.h	39
7.13 include/zmogus.h File Reference	39
7.13.1 Detailed Description	39
7.14 zmogus.h	40
7.15 src/io_deque.cpp File Reference	40
7.15.1 Function Documentation	41
7.15.1.1 benchmarkProcessingFile()	41
7.15.1.2 containsDigit()	42
7.15.1.3 duomenulrasympasFaile()	42
7.15.1.4 duomenulrasympasKonsole()	42
7.15.1.5 fileRead()	43
7.15.1.6 fileTest()	43
7.15.1.7 GenerateStudentsFile()	43
7.15.1.8 getSortChoice()	43
7.15.1.9 inputas()	44
7.15.1.10 outputas()	44
7.15.1.11 RuleOfFiveTestas()	44
7.15.1.12 sortByUser()	44
7.15.1.13 SplitStudentsStrategy1()	45
7.15.1.14 SplitStudentsStrategy2()	45
7.15.1.15 SplitStudentsStrategy3()	45
7.16 src/io_list.cpp File Reference	46
7.16.1 Function Documentation	46
7.16.1.1 benchmarkProcessingFile()	46
7.16.1.2 containsDigit()	47
7.16.1.3 duomenulrasympasFaile()	47
7.16.1.4 duomenulrasympasKonsole()	47
7.16.1.5 fileRead()	47
7.16.1.6 fileTest()	47
7.16.1.7 GenerateStudentsFile()	48
7.16.1.8 getSortChoice()	48
7.16.1.9 inputas()	48
7.16.1.10 outputas()	48
7.16.1.11 sortByUser()	48
7.16.1.12 SplitStudentsStrategy1()	48

7.16.1.13 SplitStudentsStrategy2()	49
7.16.1.14 SplitStudentsStrategy3()	49
7.17 src/io_vector.cpp File Reference	49
7.17.1 Function Documentation	50
7.17.1.1 benchmarkProcessingFile()	50
7.17.1.2 containsDigit()	50
7.17.1.3 duomenulrasymasFaile()	50
7.17.1.4 duomenulrasymasKonsole()	50
7.17.1.5 fileRead()	51
7.17.1.6 fileTest()	51
7.17.1.7 GenerateStudentsFile()	51
7.17.1.8 getSortChoice()	51
7.17.1.9 inputas()	51
7.17.1.10 outputas()	51
7.17.1.11 sortByUser()	52
7.17.1.12 SplitStudentsStrategy1()	52
7.17.1.13 SplitStudentsStrategy2()	52
7.17.1.14 SplitStudentsStrategy3()	52
7.18 src/main_deque.cpp File Reference	52
7.18.1 Function Documentation	52
7.18.1.1 main()	52
7.19 src/main_list.cpp File Reference	53
7.19.1 Function Documentation	53
7.19.1.1 main()	53
7.20 src/main_vector.cpp File Reference	53
7.20.1 Function Documentation	53
7.20.1.1 main()	53
7.21 src/student.cpp File Reference	53
7.21.1 Function Documentation	54
7.21.1.1 operator<<()	54
7.21.1.2 operator>>()	54
7.22 src/utlis.cpp File Reference	55
7.22.1 Function Documentation	55
7.22.1.1 intInput()	55
7.22.1.2 randomVardasPavarde()	56
7.22.2 Variable Documentation	56
7.22.2.1 mot_pavardes	56
7.22.2.2 vardai	56
7.22.2.3 vyr_pavardes	56

Chapter 1

Directory Hierarchy

1.1 Directories

include	9
student.h	21
student_io_deque.h	23
student_io_list.h	29
student_io_vector.h	33
timer.h	36
utils.h	37
zmogus.h	39
src	9
io_deque.cpp	40
io_list.cpp	46
io_vector.cpp	49
main_deque.cpp	52
main_list.cpp	53
main_vector.cpp	53
student.cpp	53
utils.cpp	55

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Timer	17
Zmogus	18
Studentas	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	Klase, skirta saugoti studento duomenis, paveldi Zmogus klase	11
Timer	Klase, skirta programos vykdymo laikui matuoti	17
Zmogus	Abstrakti bazine klase, sauganti bendrus zmogaus duomenis	18

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/ student.h	
Studentas klasės aprašymas	21
include/ student_io_deque.h	
Funkcijos darbui su Studentas objektais naudojant deque konteineri	23
include/ student_io_list.h	29
include/ student_io_vector.h	33
include/ timer.h	
Laiko matavimo klasės aprašymas	36
include/ utils.h	
Pagalbinių funkcijų ir duomenų aprašymas	37
include/ zmogus.h	
Zmogus abstrakčios bazinės klasės aprašymas	39
src/ io_deque.cpp	40
src/ io_list.cpp	46
src/ io_vector.cpp	49
src/ main_deque.cpp	52
src/ main_list.cpp	53
src/ main_vector.cpp	53
src/ student.cpp	53
src/ utils.cpp	55

Chapter 5

Directory Documentation

5.1 include Directory Reference

Files

- file [student.h](#)
Studentas klases aprasymas.
- file [student_io_deque.h](#)
Funkcijos darbui su *Studentas* objektais naudojant deque konteineri.
- file [student_io_list.h](#)
- file [student_io_vector.h](#)
- file [timer.h](#)
Laiko matavimo klases aprasymas.
- file [utils.h](#)
Pagalbiniu funkciju ir duomenu aprasymas.
- file [zmogus.h](#)
Zmogus abstrakcios bazines klases aprasymas.

5.2 src Directory Reference

Files

- file [io_deque.cpp](#)
- file [io_list.cpp](#)
- file [io_vector.cpp](#)
- file [main_deque.cpp](#)
- file [main_list.cpp](#)
- file [main_vector.cpp](#)
- file [student.cpp](#)
- file [utils.cpp](#)

Chapter 6

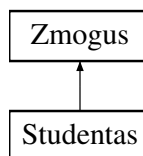
Class Documentation

6.1 Studentas Class Reference

Klase, skirta saugoti studento duomenis, paveldi [Zmogus](#) klase.

```
#include <student.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
Konstruktorius.
- [Studentas](#) (std::istream &is)
Konstruktorius, kuris nuskaito studento duomenis is ivesties srauto.
- [~Studentas](#) ()
Destruktorius.
- [Studentas](#) (const [Studentas](#) &other)
Kopijavimo konstruktorius.
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
Kopijavimo priskyrimo operatorius.
- [Studentas](#) ([Studentas](#) &&other) noexcept
Perkelimo konstruktorius.
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
Perkelimo priskyrimo operatorius.
- const vector< int > [paz](#) () const
grazina Studento Pazymius
- int [egz](#) () const
grazina Studento egzamino pazymi
- double [vid](#) () const

- grazina Studento pazymiu vidurki*
- double [med](#) () const
- grazina Studento pazymiu mediana*
- std::istream & [ReadStudent](#) (std::istream &)
- void [SetEgz](#) (int egz)
- Nustato studento egzamino pazymi.*
- void [SetVid](#) (double vid)
- Nustato galutini bala pagal vidurki.*
- void [SetMed](#) (double med)
- Nustato galutini bala pagal mediana.*
- void [AddPaz](#) (int paz)
- Prideda viena namu darbu pazymi.*
- void [ClearPaz](#) ()
- Isvalo visus namu darbu pazymius.*
- void [MedIrVidSkaciavimas](#) (int sum)
- Apskaiciuoja galutinius balus pagal vidurki ir mediana.*
- void [spausdinti](#) () const override
- Isveda studento duomenis.*

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- Konstruktorius.*
- [Zmogus](#) (const string &vardas, const string &pavarde)
- Konstruktorius su vardu ir pavarde.*
- virtual [~Zmogus](#) ()
- Destruktorius.*
- const string [vardas](#) () const
- grazina Zmogaus Vardas*
- const string [pavarde](#) () const
- grazina Zmogaus Pavarde*
- void [SetVardas](#) (const string &vardas)
- Iraso zmogaus varda.*
- void [SetPavarde](#) (const string &pavarde)
- Iraso zmogaus pavarde.*

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [_vardas](#)
- string [_pavarde](#)

6.1.1 Detailed Description

Klase, skirta saugoti studento duomenis, paveldi [Zmogus](#) klase.

[Studentas](#) klase saugo studento pazymius, egzamina, vidurki ir mediana Ji paveldi varda ir pavarde is zmogus klases

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Studentas() [1/4]

```
Studentas::Studentas () [inline]
```

Konstruktorius.

Sukuria tuscia studento objekta. Nustato pazymius 0

6.1.2.2 Studentas() [2/4]

```
Studentas::Studentas (  
    std::istream & is)
```

Konstruktorius, kuris nuskaito studento duomenis is ivesties srauto.

Parameters

<i>is</i>	Ivesties srautas, is kurio bus skaitomi studento duomenys
-----------	---

6.1.2.3 ~Studentas()

```
Studentas::~~Studentas ()
```

Destruktorius.

6.1.2.4 Studentas() [3/4]

```
Studentas::Studentas (  
    const Studentas & other)
```

Kopijavimo konstruktorius.

Parameters

<i>other</i>	<code>Studentas</code> objektas, is kurio kopijuojami duomenys.
--------------	---

6.1.2.5 Studentas() [4/4]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Perkelimo konstruktorius.

Parameters

<i>other</i>	<code>Studentas</code> objektas, is kurio perkeliami duomenys.
--------------	--

6.1.3 Member Function Documentation

6.1.3.1 AddPaz()

```
void Studentas::AddPaz (  
    int paz) [inline]
```

Prideda viena namu darbu pazymi.

Parameters

<i>paz</i>	Namu darbu pazymys.
------------	---------------------

6.1.3.2 ClearPaz()

```
void Studentas::ClearPaz () [inline]
```

Isvalo visus namu darbu pazymius.

6.1.3.3 egz()

```
int Studentas::egz () const [inline]
```

grazina Studento egzamino pazymi

Returns

Studento egzamino pazymis

6.1.3.4 med()

```
double Studentas::med () const [inline]
```

grazina Studento pazymiu mediana

Returns

Studento pazymiu mediana

6.1.3.5 MedIrVidSkaciavimas()

```
void Studentas::MedIrVidSkaciavimas (
    int sum)
```

Apskaiciuoja galutinius balus pagal vidurki ir mediana.

Parameters

<i>sum</i>	Namu darbu pazymiu suma.
------------	--------------------------

6.1.3.6 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other)
```

Kopijavimo priskyrimo operatorius.

Parameters

<i>other</i>	Studentas objektas, is kurio kopijuojami duomenys.
--------------	--

Returns

Nuoroda i si objekta.

6.1.3.7 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other) [noexcept]
```

Perkelimo priskyrimo operatorius.

Parameters

<i>other</i>	Studentas objektas, is kurio perkeliama duomenys.
--------------	---

Returns

Nuoroda i si objekta.

6.1.3.8 paz()

```
const vector< int > Studentas::paz () const [inline]
```

grazina Studento Pazymius

Returns

Studento pazymius vector<int>

6.1.3.9 ReadStudent()

```
std::istream & Studentas::ReadStudent (
    std::istream & is)
```

6.1.3.10 SetEgz()

```
void Studentas::SetEgz (
    int egz) [inline]
```

Nustato studento egzamino pazymi.

Parameters

<i>egz</i>	Egzamino pazymys.
------------	-------------------

6.1.3.11 SetMed()

```
void Studentas::SetMed (
    double med) [inline]
```

Nustato galutini bala pagal mediana.

Parameters

<i>med</i>	Galutinis balas pagal mediana.
------------	--------------------------------

6.1.3.12 SetVid()

```
void Studentas::SetVid (
    double vid) [inline]
```

Nustato galutini bala pagal vidurki.

Parameters

<i>vid</i>	Galutinis balas pagal vidurki.
------------	--------------------------------

6.1.3.13 spausdinti()

```
void Studentas::spausdinti () const [override], [virtual]
```

Isveda studento duomenis.

Implements [Zmogus](#).

6.1.3.14 vid()

```
double Studentas::vid () const [inline]
```

grazina Studento pazymiu vidurki

Returns

Studento pazymiu vidurkis

The documentation for this class was generated from the following files:

- include/[student.h](#)
- src/[student.cpp](#)

6.2 Timer Class Reference

Klase, skirta programos vykdymo laikui matuoti.

```
#include <timer.h>
```

Public Member Functions

- [Timer](#) ()
Sukria laikmati ir pradeda laiko matavima.
- void [reset](#) ()
Atnaujina laikmati.
- double [elapsed](#) () const
Grazina kiek laiko praejo.

6.2.1 Detailed Description

Klase, skirta programos vykdymo laikui matuoti.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 Timer()

```
Timer::Timer () [inline]
```

Sukria laikmati ir pradeda laiko matavima.

6.2.3 Member Function Documentation

6.2.3.1 elapsed()

```
double Timer::elapsed () const [inline]
```

Grazina kiek laiko praejo.

Returns

Praeitas laikas

6.2.3.2 reset()

```
void Timer::reset () [inline]
```

Atnaujina laikmatį.

The documentation for this class was generated from the following file:

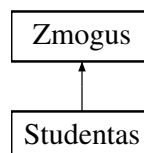
- [include/timer.h](#)

6.3 Zmogus Class Reference

Abstrakti bazinė klase, sauganti bendrus zmogaus duomenis.

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus \(\)](#)
Konstruktorius.
- [Zmogus \(const string &vardas, const string &pavarde\)](#)
Konstruktorius su vardu ir pavarde.
- virtual [~Zmogus \(\)](#)
Destruktorius.
- virtual void [spausdinti \(\)](#) const =0
- const string [vardas \(\)](#) const
grazina Zmogaus Vardas
- const string [pavarde \(\)](#) const
grazina Zmogaus Pavarde
- void [SetVardas \(const string &vardas\)](#)
Iraso zmogaus varda.
- void [SetPavarde \(const string &pavarde\)](#)
Iraso zmogaus pavarde.

Protected Attributes

- string [_vardas](#)
- string [_pavarde](#)

6.3.1 Detailed Description

Abstrakti bazine klase, sauganti bendrus zmogaus duomenis.

Sia klase paveldi [Studentas](#) klase Is jos negalima kurti objektu tiesiogiai [Zmogus](#) klase yra abstrakti, nes turi metoda [spausdinti\(\)](#), kuri [Studentas](#) klase turi perrasyti.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

Konstruktorius.

6.3.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (  
    const string & vardas,  
    const string & pavarde) [inline]
```

Konstruktorius su vardu ir pavarde.

Parameters

<i>vardas</i>	Zmogus vardas
<i>pavarde</i>	Zmogus pavarde

6.3.2.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Destruktorius.

6.3.3 Member Function Documentation

6.3.3.1 pavarde()

```
const string Zmogus::pavarde () const [inline]
```

grazina Zmogaus Pavarde

Returns

Zmogaus pavarde

6.3.3.2 SetPavarde()

```
void Zmogus::SetPavarde (
    const string & pavarde) [inline]
```

Iraso zmogaus pavarde.

Parameters

<i>pavarde</i>	naujas Pavarde
----------------	----------------

6.3.3.3 SetVardas()

```
void Zmogus::SetVardas (
    const string & vardas) [inline]
```

Iraso zmogaus vardas.

Parameters

<i>vardas</i>	naujas Vardas
---------------	---------------

6.3.3.4 spausdinti()

```
virtual void Zmogus::spausdinti () const [pure virtual]
```

Implemented in [Studentas](#).

6.3.3.5 vardas()

```
const string Zmogus::vardas () const [inline]
```

grazina Zmogaus Vardas

Returns

Zmogaus vardas

6.3.4 Member Data Documentation

6.3.4.1 _pavarde

```
string Zmogus::_pavarde [protected]
```

Pavarde

6.3.4.2 _vardas

```
string Zmogus::_vardas [protected]
```

Vardas

The documentation for this class was generated from the following file:

- include/[zmogus.h](#)

Chapter 7

File Documentation

7.1 include/student.h File Reference

[Studentas](#) klases aprasymas.

```
#include <string>
#include <vector>
#include <iostream>
#include "zmogus.h"
```

Classes

- class [Studentas](#)

Klase, skirta saugoti studento duomenis, paveldi [Zmogus](#) klase.

Functions

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
Ivesties operatorius studento duomenims nuskaityti.
- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)
Isvesties operatorius studento duomenims isvesti.

7.1.1 Detailed Description

[Studentas](#) klases aprasymas.

7.1.2 Function Documentation

7.1.2.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s)
```

Išvesties operatorius studento duomenims išvesti.

Parameters

<i>os</i>	Išvesties srautas.
<i>s</i>	<code>Studentas</code> objektas.

Returns

Išvesties srautas.

7.1.2.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s)
```

Išvesties operatorius studento duomenims nuskaityti.

Parameters

<i>is</i>	Išvesties srautas.
<i>s</i>	<code>Studentas</code> objektas.

Returns

Išvesties srautas.

7.2 student.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENT_H
00002 #define STUDENT_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <iostream>
00007 #include "zmogus.h"
00008
00009 using std::string;
00010 using std::vector;
00011
00016
00024
00025 class Studentas : public Zmogus
```

```

00026 {
00027 private:
00028     vector<int> _paz;
00029     int _egz;
00030     double _vid;
00031     double _med;
00032
00033 public:
00040     Studentas() : Zmogus(), _egz(0), _vid(0.0), _med(0.0) {}
00046     Studentas(std::istream &is);
00047
00051     ~Studentas(); // destruktorius
00056     Studentas(const Studentas &other); // copy konstruktorius - sukuria nauja objekta
00062     Studentas &operator=(const Studentas &other); // copy priskyrimo konstruktorius - kopijuojame
    egzistuojancia objekta
00067     Studentas(Studentas &&other) noexcept; // noexcept - neturetu mesti isimciu
00073     Studentas &operator=(Studentas &&other) noexcept; // noexcept - neturetu mesti isimciu
00074
00079     inline const vector<int> paz() const { return _paz; }
00084     inline int egz() const { return _egz; }
00089     inline double vid() const { return _vid; }
00094     inline double med() const { return _med; }
00095
00096     std::istream &ReadStudent(std::istream &);
00097
00102     void SetEgz(int egz) { _egz = egz; }
00107     void SetVid(double vid) { _vid = vid; }
00112     void SetMed(double med) { _med = med; }
00117     void AddPaz(int paz) { _paz.push_back(paz); }
00121     void ClearPaz() { _paz.clear(); }
00122
00127     void MedIrVidSkaciavimas(int sum);
00128
00132     void spausdinti() const override;
00133 };
00134
00141 std::istream& operator>(std::istream& is, Studentas& s);
00142
00149 std::ostream& operator<(std::ostream& os, const Studentas& s);
00150
00151 #endif

```

7.3 include/student_io_deque.h File Reference

Funkcijos darbu su `Studentas` objektais naudojant deque konteineri.

```

#include "student.h"
#include <string>
#include <deque>

```

Functions

- void `inputas` (deque< `Studentas` > &grupe)
Leidžia vartotojui ivesti arba nuskaityti studento duomenis.
- void `outputas` (deque< `Studentas` > &grupe)
Leidžia vartotojui pasirinkti studentu isvedimo buda.
- void `fileRead` (deque< `Studentas` > &grupe, string file_name)
Nuskaito studentu duomenis is failo.
- void `fileTest` (deque< `Studentas` > &grupe, string file_name, int &testKiekis)
Testuoja studentu nuskaityma is failo kelis kartus.
- void `sortByUser` (deque< `Studentas` > &grupe, int temp)
Grazina vartotojo pasirinkta rikiavimo kriteriju.
- void `duomenulrasymasFaile` (const deque< `Studentas` > &grupe, int temp, const string &fileName)
Iraso studentu duomenis i faila.
- void `duomenulrasymasKonsole` (deque< `Studentas` > &grupe, int temp)

- Isveda studentu duomenis i konsolę.*
- bool `containsDigit` (const string &str)
Patikrina, ar eilutę sudaryta tik iš skaitmenų.
- void `GenerateStudentsFile` (int n)
Sugeneruoja studentų duomenų failą.
- void `SplitStudentsStrategy1` (const deque< `Studentas` > &grupe, deque< `Studentas` > &failed, deque< `Studentas` > &passed)
Padalina studentus į dvi grupes: neįslaukusius ir įslaukusius.
- void `SplitStudentsStrategy2` (deque< `Studentas` > &grupe, deque< `Studentas` > &failed)
Padalina studentus į dvi grupes, paliekant įslaukusius pradinėje grupėje.
- void `SplitStudentsStrategy3` (deque< `Studentas` > &grupe, deque< `Studentas` > &failed)
Padalina studentus į dvi grupes naudojant stable_partition.
- int `getSortChoice` (int temp)
Rusiuoja pagal vartotojo pasirinktą variantą.
- void `benchmarkProcessingFile` (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greičio testą.
- void `RuleOfFiveTestas` ()
Atlieka Rule of Five demonstracinį testą `Studentas` klasei.

7.3.1 Detailed Description

Funkcijos darbui su `Studentas` objektais naudojant deque konteinerį.

7.3.2 Function Documentation

7.3.2.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greičio testą.

Parameters

<i>n</i>	Studentų skaičius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentų skirstymo strategijos numeris.

7.3.2.2 containsDigit()

```
bool containsDigit (  
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

7.3.2.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (  
    const deque< Studentas > & grupe,  
    int temp,  
    const string & fileName)
```

Iraso studentu duomenis i faila.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.
<i>fileName</i>	Failo pavadinimas.

7.3.2.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (  
    deque< Studentas > & grupe,  
    int temp)
```

Isveda studentu duomenis i konsole.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.

7.3.2.5 fileRead()

```
void fileRead (
    deque< Studentas > & grupe,
    string file_name)
```

Nuskaito studentu duomenis is failo.

Parameters

<i>grupe</i>	Studentu deque konteineris, i kuri irasomi nuskaityti studentai.
<i>file_name</i>	Failo pavadinimas.

7.3.2.6 fileTest()

```
void fileTest (
    deque< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

Testuoja studentu nuskaityma is failo kelis kartus.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>file_name</i>	Failo pavadinimas.
<i>testKiekis</i>	Testu skaicius.

7.3.2.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomenu faila.

Parameters

<i>n</i>	Studentu skaicius faile.
----------	--------------------------

7.3.2.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinkta variantą.

Parameters

<i>temp</i>	Rusiavimo formato pasirinkimas.
-------------	---------------------------------

7.3.2.9 inputas()

```
void inputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui iversti arba nuskaityti studento duomenis.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

7.3.2.10 outputas()

```
void outputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui pasirinkti studentu isvedimo buda.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

7.3.2.11 RuleOfFiveTestas()

```
void RuleOfFiveTestas ()
```

Atlieka Rule of Five demonstracini testa [Studentas](#) klasei.

7.3.2.12 sortByUser()

```
void sortByUser (  
    deque< Studentas > & grupe,  
    int temp)
```

Grazina vartotojo pasirinkta rikiavimo kriteriju.

Parameters

<i>temp</i>	Pasirinktas isvedimo formatas.
-------------	--------------------------------

Returns

Rikiavimo pasirinkimo numeris.

7.3.2.13 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (
    const deque< Studentas > & grupe,
    deque< Studentas > & failed,
    deque< Studentas > & passed)
```

Padalina studentus i dvi grupes: neislaikiusius ir islaikiusius.

Parameters

<i>grupe</i>	Pradine studentu grupe.
<i>failed</i>	Neislaikiusiu studentu grupe.
<i>passed</i>	Islaikiusiu studentu grupe.

7.3.2.14 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    deque< Studentas > & grupe,
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes, paliekant islaikiusius pradineje grupeje.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

7.3.2.15 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    deque< Studentas > & grupe,
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes naudojant `stable_partition`.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

7.4 student_io_deque.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STUDENT_IO_H_DEQUE
00002 #define STUDENT_IO_H_DEQUE
00003
00004 #include "student.h"
00005 #include <string>
00006 #include <deque>
00007
00008 using std::deque;
00009 using std::string;
00010
00015
00020 void inputas(deque<Studentas> &grupe);
00021
00026 void outputas(deque<Studentas>& grupe);
00027
00033 void fileRead(deque<Studentas> &grupe, string file_name);
00040 void fileTest(deque<Studentas> &grupe, string file_name, int &testKiekis);
00046 void sortByUser(deque<Studentas> &grupe, int temp);
00053 void duomenulrasyamasFaile(const deque<Studentas>& grupe, int temp, const string& fileName);
00059 void duomenulrasyamasKonsole(deque<Studentas> &grupe, int temp);
00060
00066 bool containsDigit(const string &str);
00071 void GenerateStudentsFile(int n);
00078 void SplitStudentsStrategy1(const deque<Studentas> &grupe, deque<Studentas> &failed, deque<Studentas>
    &passed);
00084 void SplitStudentsStrategy2(deque<Studentas> &grupe, deque<Studentas> &failed);
00090 void SplitStudentsStrategy3(deque<Studentas> &grupe, deque<Studentas> &failed);
00091
00092
00097 int getSortChoice(int temp);
00098
00105 void benchmarkProcessingFile(int n, int testKiekis, int strategy);
00106
00110 void RuleOfFiveTestas();
00111
00112 #endif

```

7.5 include/student_io_list.h File Reference

```

#include "student.h"
#include <string>
#include <list>

```

Functions

- void [inputas](#) (list< [Studentas](#) > &grupe)
 - void [outputas](#) (list< [Studentas](#) > &grupe)
 - void [fileRead](#) (list< [Studentas](#) > &grupe, string file_name)
 - void [fileTest](#) (list< [Studentas](#) > &grupe, string file_name, int &testKiekis)
 - void [sortByUser](#) (list< [Studentas](#) > &grupe, int temp)
 - void [duomenulrasyamasFaile](#) (list< [Studentas](#) > &grupe, int temp, string fileName)
 - void [duomenulrasyamasKonsole](#) (list< [Studentas](#) > &grupe, int temp)
 - bool [containsDigit](#) (const string &str)
- Patikrina, ar eilute sudaryta tik is skaitmenu.*
- void [GenerateStudentsFile](#) (int n)
- Sugeneruoja studentu duomeniu faila.*
- void [SplitStudentsStrategy1](#) (const list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed, list< [Studentas](#) > &passed)
 - void [SplitStudentsStrategy2](#) (list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed)
 - void [SplitStudentsStrategy3](#) (list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed)

- int `getSortChoice` (int temp)
Rusiuoja pagal vartotojo pasirinkta varianta.
- void `benchmarkProcessingFile` (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

7.5.1 Function Documentation

7.5.1.1 `benchmarkProcessingFile()`

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

7.5.1.2 `containsDigit()`

```
bool containsDigit (
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

7.5.1.3 `duomenulrasymasFaile()`

```
void duomenulrasymasFaile (
    list< Studentas > & grupe,
    int temp,
    string fileName)
```

7.5.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (
    list< Studentas > & grupe,
    int temp)
```

7.5.1.5 fileRead()

```
void fileRead (
    list< Studentas > & grupe,
    string file_name)
```

7.5.1.6 fileTest()

```
void fileTest (
    list< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

7.5.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomeni faila.

Parameters

<i>n</i>	Studentu skaicius faile.
----------	--------------------------

7.5.1.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinkta varianta.

Parameters

<i>temp</i>	Rusiavimo formato pasirinkimas.
-------------	---------------------------------

7.5.1.9 inputas()

```
void inputas (
    list< Studentas > & grupe)
```

7.5.1.10 outputas()

```
void outputas (
    list< Studentas > & grupe)
```

7.5.1.11 sortByUser()

```
void sortByUser (
    list< Studentas > & grupe,
    int temp)
```

7.5.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (
    const list< Studentas > & grupe,
    list< Studentas > & failed,
    list< Studentas > & passed)
```

7.5.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

7.5.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

7.6 student_io_list.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENT_IO_H_LIST
00002 #define STUDENT_IO_H_LIST
00003
00004 #include "student.h"
00005 #include <string>
00006 #include <list>
00007
00008 using std::string;
00009 using std::list;
00010
00011 void inputas(list<Studentas> &grupe);
00012 void outputas(list<Studentas> &grupe);
00013
00014 void fileRead(list<Studentas> &grupe, string file_name);
00015 void fileTest(list<Studentas> &grupe, string file_name, int &testKiekis);
00016
00017 void sortByUser(list<Studentas> &grupe, int temp);
00018
00019 void duomenuIrasymasFaile(list<Studentas> &grupe, int temp, string fileName);
00020 void duomenuIrasymasKonsole(list<Studentas> &grupe, int temp);
00021
00022 bool containsDigit(const string &str);
00023 void GenerateStudentsFile(int n);
00024 void SplitStudentsStrategy1(const list<Studentas> &grupe, list<Studentas> &failed, list<Studentas>
    &passed);
00025 void SplitStudentsStrategy2(list<Studentas> &grupe, list<Studentas> &failed);
00026 void SplitStudentsStrategy3(list<Studentas> &grupe, list<Studentas> &failed);
00027
00028 int getSortChoice(int temp);
00029
00030 void benchmarkProcessingFile(int n, int testKiekis, int strategy);
00031
00032 #endif
```


7.7 include/student_io_vector.h File Reference

```
#include "student.h"
#include <string>
#include <vector>
```

Functions

- void [inputas](#) (vector< [Studentas](#) > &grupe)
- void [outputas](#) (vector< [Studentas](#) > &grupe)
- void [fileRead](#) (vector< [Studentas](#) > &grupe, string file_name)
- void [fileTest](#) (vector< [Studentas](#) > &grupe, string file_name, int &testKiekis)
- void [sortByUser](#) (vector< [Studentas](#) > &grupe, int temp)
- void [duomenulrasymasFaile](#) (vector< [Studentas](#) > &grupe, int temp, string fileName)
- void [duomenulrasymasKonsole](#) (vector< [Studentas](#) > &grupe, int temp)
- bool [containsDigit](#) (const string &str)
Patikrina, ar eilute sudaryta tik is skaitmenu.
- void [GenerateStudentsFile](#) (int n)
Sugeneruoja studentu duomeniu faila.
- void [SplitStudentsStrategy1](#) (const vector< [Studentas](#) > &grupe, vector< [Studentas](#) > &failed, vector< [Studentas](#) > &passed)
- void [SplitStudentsStrategy2](#) (vector< [Studentas](#) > &grupe, vector< [Studentas](#) > &failed)
- void [SplitStudentsStrategy3](#) (vector< [Studentas](#) > &grupe, vector< [Studentas](#) > &failed)
- int [getSortChoice](#) (int temp)
Rusiuoja pagal vartotojo pasirinkta varianta.
- void [benchmarkProcessingFile](#) (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

7.7.1 Function Documentation

7.7.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

7.7.1.2 containsDigit()

```
bool containsDigit (
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

7.7.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (
    vector< Studentas > & grupe,
    int temp,
    string fileName)
```

7.7.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (
    vector< Studentas > & grupe,
    int temp)
```

7.7.1.5 fileRead()

```
void fileRead (
    vector< Studentas > & grupe,
    string file_name)
```

7.7.1.6 fileTest()

```
void fileTest (
    vector< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

7.7.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (  
    int n)
```

Sugeneruoja studentu duomenų failą.

Parameters

<i>n</i>	Studentų skaičius failė.
----------	--------------------------

7.7.1.8 getSortChoice()

```
int getSortChoice (  
    int temp)
```

Rusiuoja pagal vartotojo pasirinktą variantą.

Parameters

<i>temp</i>	Rusavimo formato pasirinkimas.
-------------	--------------------------------

7.7.1.9 inputas()

```
void inputas (  
    vector< Studentas > & grupe)
```

7.7.1.10 outputas()

```
void outputas (  
    vector< Studentas > & grupe)
```

7.7.1.11 sortByUser()

```
void sortByUser (  
    vector< Studentas > & grupe,  
    int temp)
```

7.7.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (  
    const vector< Studentas > & grupe,  
    vector< Studentas > & failed,  
    vector< Studentas > & passed)
```

7.7.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

7.7.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

7.8 student_io_vector.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENT_IO_H_VECTOR
00002 #define STUDENT_IO_H_VECTOR
00003
00004 #include "student.h"
00005 #include <string>
00006 #include <vector>
00007
00008 using std::vector;
00009 using std::string;
00010
00011
00012 void inputas(vector<Studentas> &grupe);
00013 void outputas(vector<Studentas> &grupe);
00014
00015 void fileRead(vector<Studentas> &grupe, string file_name);
00016 void fileTest(vector<Studentas> &grupe, string file_name, int &testKiekis);
00017
00018 void sortByUser(vector<Studentas> &grupe, int temp);
00019
00020 void duomenuIrasymasFaile(vector<Studentas> &grupe, int temp, string fileName);
00021 void duomenuIrasymasKonsole(vector<Studentas> &grupe, int temp);
00022
00023 bool containsDigit(const string &str);
00024 void GenerateStudentsFile(int n);
00025
00026 void SplitStudentsStrategy1(const vector<Studentas> &grupe, vector<Studentas> &failed,
    vector<Studentas> &passed);
00027 void SplitStudentsStrategy2(vector<Studentas> &grupe, vector<Studentas> &failed);
00028 void SplitStudentsStrategy3(vector<Studentas> &grupe, vector<Studentas> &failed);
00029
00030 int getSortChoice(int temp);
00031
00032 void benchmarkProcessingFile(int n, int testKiekis, int strategy);
00033
00034 #endif
```

7.9 include/timer.h File Reference

Laiko matavimo klases aprasymas.

```
#include <chrono>
```

Classes

- class [Timer](#)

Klase, skirta programos vykdymo laikui matuoti.

7.9.1 Detailed Description

Laiko matavimo klases aprasymas.

7.10 timer.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include <chrono>
00005
00010
00015
00016 class Timer
00017 {
00018     // usage of using
00019     using hrClock = std::chrono::high_resolution_clock;
00021     using durationDouble = std::chrono::duration<double>;
00022
00023 private:
00024     std::chrono::time_point<hrClock> start;
00025
00026 public:
00030     Timer() : start{hrClock::now()} {}
00034     void reset()
00035     {
00036         start = hrClock::now();
00037     }
00042     double elapsed() const
00043     {
00044         return durationDouble(hrClock::now() - start).count();
00045     }
00046 };
00047
00048 #endif
```

7.11 include/utils.h File Reference

Pagalbiniu funkciju ir duomenu aprasymas.

```
#include <string>
#include <vector>
```

Functions

- void [intInput](#) (int &temp)
Nuskaityto sveikaji skaiciu ir patikrina ivesti.
- void [randomVardasPavarde](#) (string &vardas, string &pavarde)
Sugeneruoja atsitiktini varda ir pavarde.

Variables

- const vector< string > [vardai](#)
Vardu sarasas atsitiktiniam generavimui.
- const vector< string > [vyr_pavardes](#)
Vyrisku pavardziu sarasas atsitiktiniam generavimui.
- const vector< string > [mot_pavardes](#)
Moterisku pavardziu sarasas atsitiktiniam generavimui.

7.11.1 Detailed Description

Pagalbiniu funkciju ir duomenu aprasymas.

7.11.2 Function Documentation

7.11.2.1 intInput()

```
void intInput (  
    int & temp)
```

Nuskaito sveikaji skaiciu ir patikrina ivesti.

Parameters

<i>temp</i>	Kintamasis, i kuri irasomas nuskaitytas skaicius.
-------------	---

7.11.2.2 randomVardasPavarde()

```
void randomVardasPavarde (  
    string & vardas,  
    string & pavarde)
```

Sugeneruoja atsitiktini varda ir pavarde.

Parameters

<i>vardas</i>	Sugeneruotas vardas.
<i>pavarde</i>	Sugeneruota pavarde.

7.11.3 Variable Documentation

7.11.3.1 mot_pavardes

```
const vector<string> mot_pavardes [extern]
```

Moterisku pavardziu sarasas atsitiktiniam generavimui.

7.11.3.2 vardai

```
const vector<string> vardai [extern]
```

Vardu sarasas atsitiktiniam generavimui.

7.11.3.3 vyr_pavardes

```
const vector<string> vyr_pavardes [extern]
```

Vyriskų pavardžių sąrašas atsitiktiniam generavimui.

7.12 utils.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UTILS_H
00002 #define UTILS_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 using std::vector;
00008 using std::string;
00009
00014
00018 extern const vector<string> vardai;
00019
00023 extern const vector<string> vyr_pavardes;
00024
00028 extern const vector<string> mot_pavardes;
00029
00034 void intInput(int& temp);
00035
00041 void randomVardasPavarde(string& vardas, string& pavarde);
00042
00043 #endif
```

7.13 include/zmogus.h File Reference

[Zmogus](#) abstrakcijos bazinės klasės aprašymas.

```
#include <string>
```

Classes

- class [Zmogus](#)
Abstrakti bazinė klasė, sauganti bendrus žmogaus duomenis.

7.13.1 Detailed Description

[Zmogus](#) abstrakcijos bazinės klasės aprašymas.

7.14 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00006 using std::string;
00007
00012
00022
00023 class Zmogus
00024 {
00025 protected:
00026     string _vardas;
00027     string _pavarde;
00028
00029 public:
00033     Zmogus() {}
00039     Zmogus(const string &vardas, const string &pavarde)
00040         : _vardas(vardas), _pavarde(pavarde) {}
00044     virtual ~Zmogus() {}
00045
00046     virtual void spausdinti() const = 0;
00051     inline const string vardas() const { return _vardas; }
00056     inline const string pavarde() const { return _pavarde; }
00061     void SetVardas(const string &vardas) { _vardas = vardas; }
00066     void SetPavarde(const string &pavarde) { _pavarde = pavarde; }
00067 };
00068
00069 #endif

```

7.15 src/io_deque.cpp File Reference

```

#include "student_io_deque.h"
#include "utils.h"
#include "timer.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <ctime>
#include <stdlib.h>
#include <utility>

```

Functions

- void **outputas** (deque< **Studentas** > &grupe)
Leidžia vartotojui pasirinkti studentu isvedimo buda.
- void **inputas** (deque< **Studentas** > &grupe)
Leidžia vartotojui iversti arba nuskaityti studento duomenis.
- void **fileTest** (deque< **Studentas** > &grupe, string file_name, int &testKiekis)
Testuoja studentu nuskaityma is failo kelis kartus.
- void **fileRead** (deque< **Studentas** > &grupe, string file_name)
Nuskaityti studentu duomenis is failo.
- void **sortByUser** (deque< **Studentas** > &grupe, int temp)

- Grazina vartotojo pasirinkta rikiavimo kriteriju.*
- int [getSortChoice](#) (int temp)
 - Rusiuoja pagal vartotojo pasirinkta varianta.*
- void [duomenulrasymasFaile](#) (const deque< [Studentas](#) > &grupe, int temp, const string &fileName)
 - Iraso studentu duomenis i faila.*
- void [duomenulrasymasKonsole](#) (deque< [Studentas](#) > &grupe, int temp)
 - Isveda studentu duomenis i konsole.*
- bool [containsDigit](#) (const string &str)
 - Patikrina, ar eilute sudaryta tik is skaitmenu.*
- void [GenerateStudentsFile](#) (int n)
 - Sugeneruoja studentu duomeniu faila.*
- void [SplitStudentsStrategy1](#) (const deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed, deque< [Studentas](#) > &passed)
 - Padalina studentus i dvi grupes: neislaikiusius ir islaikiusius.*
- void [SplitStudentsStrategy2](#) (deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed)
 - Padalina studentus i dvi grupes, paliekant islaikiusius pradineje grupeje.*
- void [SplitStudentsStrategy3](#) (deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed)
 - Padalina studentus i dvi grupes naudojant stable_partition.*
- void [benchmarkProcessingFile](#) (int n, int testKiekis, int strategy)
 - Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.*
- void [RuleOfFiveTestas](#) ()
 - Atlieka Rule of Five demonstracini testa [Studentas](#) klasei.*

7.15.1 Function Documentation

7.15.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

7.15.1.2 containsDigit()

```
bool containsDigit (
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

7.15.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (
    const deque< Studentas > & grupe,
    int temp,
    const string & fileName)
```

Iraso studentu duomenis i faila.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.
<i>fileName</i>	Failo pavadinimas.

7.15.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (
    deque< Studentas > & grupe,
    int temp)
```

Isveda studentu duomenis i konsole.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.

7.15.1.5 fileRead()

```
void fileRead (
    deque< Studentas > & grupe,
    string file_name)
```

Nuskaito studentu duomenis is failo.

Parameters

<i>grupe</i>	Studentu deque konteineris, i kuri irasomi nuskaityti studentai.
<i>file_name</i>	Failo pavadinimas.

7.15.1.6 fileTest()

```
void fileTest (
    deque< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

Testuoja studentu nuskaityma is failo kelis kartus.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>file_name</i>	Failo pavadinimas.
<i>testKiekis</i>	Testu skaicius.

7.15.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomenu faila.

Parameters

<i>n</i>	Studentu skaicius faile.
----------	--------------------------

7.15.1.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinkta varianta.

Parameters

<i>temp</i>	Rusiavimo formato pasirinkimas.
-------------	---------------------------------

7.15.1.9 inputas()

```
void inputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui iversti arba nuskaityti studento duomenis.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

7.15.1.10 outputas()

```
void outputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui pasirinkti studentu isvedimo buda.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

7.15.1.11 RuleOfFiveTestas()

```
void RuleOfFiveTestas ()
```

Atlieka Rule of Five demonstracini testa [Studentas](#) klasei.

7.15.1.12 sortByUser()

```
void sortByUser (  
    deque< Studentas > & grupe,  
    int temp)
```

Grazina vartotojo pasirinkta rikiavimo kriteriju.

Parameters

<i>temp</i>	Pasirinktas isvedimo formatas.
-------------	--------------------------------

Returns

Rikiavimo pasirinkimo numeris.

7.15.1.13 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (
    const deque< Studentas > & grupe,
    deque< Studentas > & failed,
    deque< Studentas > & passed)
```

Padalina studentus i dvi grupes: neislaikiusius ir islaikiusius.

Parameters

<i>grupe</i>	Pradine studentu grupe.
<i>failed</i>	Neislaikiusiu studentu grupe.
<i>passed</i>	Islaikiusiu studentu grupe.

7.15.1.14 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    deque< Studentas > & grupe,
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes, paliekant islaikiusius pradineje grupeje.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

7.15.1.15 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    deque< Studentas > & grupe,
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes naudojant `stable_partition`.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

7.16 src/io_list.cpp File Reference

```
#include "student_io_list.h"
#include "utils.h"
#include "timer.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <ctime>
#include <stdlib.h>
```

Functions

- void [outputas](#) (list< [Studentas](#) > &grupe)
- void [inputas](#) (list< [Studentas](#) > &grupe)
- void [fileTest](#) (list< [Studentas](#) > &grupe, string file_name, int &testKiekis)
- void [fileRead](#) (list< [Studentas](#) > &grupe, string file_name)
- void [sortByUser](#) (list< [Studentas](#) > &grupe, int temp)
- int [getSortChoice](#) (int temp)
Rusiuoja pagal vartotojo pasirinkta varianta.
- void [duomenulrasymasFaile](#) (list< [Studentas](#) > &grupe, int temp, string fileName)
- void [duomenulrasymasKonsole](#) (list< [Studentas](#) > &grupe, int temp)
- bool [containsDigit](#) (const string &str)
Patikrina, ar eilute sudaryta tik is skaitmenu.
- void [GenerateStudentsFile](#) (int n)
Sugeneruoja studentu duomeni faila.
- void [SplitStudentsStrategy1](#) (const list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed, list< [Studentas](#) > &passed)
- void [SplitStudentsStrategy2](#) (list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed)
- void [SplitStudentsStrategy3](#) (list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed)
- void [benchmarkProcessingFile](#) (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

7.16.1 Function Documentation

7.16.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

7.16.1.2 containsDigit()

```
bool containsDigit (  
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

7.16.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (  
    list< Studentas > & grupe,  
    int temp,  
    string fileName)
```

7.16.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (  
    list< Studentas > & grupe,  
    int temp)
```

7.16.1.5 fileRead()

```
void fileRead (  
    list< Studentas > & grupe,  
    string file_name)
```

7.16.1.6 fileTest()

```
void fileTest (  
    list< Studentas > & grupe,  
    string file_name,  
    int & testKiekis)
```

7.16.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (  
    int n)
```

Sugeneruoja studentu duomenų failą.

Parameters

<i>n</i>	Studentų skaičius failė.
----------	--------------------------

7.16.1.8 getSortChoice()

```
int getSortChoice (  
    int temp)
```

Rusiuoja pagal vartotojo pasirinktą variantą.

Parameters

<i>temp</i>	Rusavimo formato pasirinkimas.
-------------	--------------------------------

7.16.1.9 inputas()

```
void inputas (  
    list< Studentas > & grupe)
```

7.16.1.10 outputas()

```
void outputas (  
    list< Studentas > & grupe)
```

7.16.1.11 sortByUser()

```
void sortByUser (  
    list< Studentas > & grupe,  
    int temp)
```

7.16.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (  
    const list< Studentas > & grupe,  
    list< Studentas > & failed,  
    list< Studentas > & passed)
```


7.16.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

7.16.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

7.17 src/io_vector.cpp File Reference

```
#include "student_io_vector.h"
#include "utils.h"
#include "timer.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <ctime>
#include <stdlib.h>
```

Functions

- void **outputas** (vector< Studentas > &grupe)
- void **inputas** (vector< Studentas > &grupe)
- void **fileTest** (vector< Studentas > &grupe, string file_name, int &testKiekis)
- void **fileRead** (vector< Studentas > &grupe, string file_name)
- void **sortByUser** (vector< Studentas > &grupe, int temp)
- int **getSortChoice** (int temp)

Rusiuoja pagal vartotojo pasirinkta varianta.

- void **duomenulrasymasFaile** (vector< Studentas > &grupe, int temp, string fileName)
- void **duomenulrasymasKonsole** (vector< Studentas > &grupe, int temp)
- bool **containsDigit** (const string &str)

Patikrina, ar eilute sudaryta tik is skaitmenu.

- void **GenerateStudentsFile** (int n)

Sugeneruoja studentu duomenu faila.

- void **SplitStudentsStrategy1** (const vector< Studentas > &grupe, vector< Studentas > &failed, vector< Studentas > &passed)
- void **SplitStudentsStrategy2** (vector< Studentas > &grupe, vector< Studentas > &failed)
- void **SplitStudentsStrategy3** (vector< Studentas > &grupe, vector< Studentas > &failed)
- void **benchmarkProcessingFile** (int n, int testKiekis, int strategy)

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

7.17.1 Function Documentation

7.17.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (  
    int n,  
    int testKiekis,  
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

7.17.1.2 containsDigit()

```
bool containsDigit (  
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

7.17.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (  
    vector< Studentas > & grupe,  
    int temp,  
    string fileName)
```

7.17.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (  
    vector< Studentas > & grupe,  
    int temp)
```

7.17.1.5 fileRead()

```
void fileRead (
    vector< Studentas > & grupe,
    string file_name)
```

7.17.1.6 fileTest()

```
void fileTest (
    vector< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

7.17.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomenų failą.

Parameters

<i>n</i>	Studentų skaičius failą.
----------	--------------------------

7.17.1.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinktą variantą.

Parameters

<i>temp</i>	Rusavimo formato pasirinkimas.
-------------	--------------------------------

7.17.1.9 inputas()

```
void inputas (
    vector< Studentas > & grupe)
```

7.17.1.10 outputas()

```
void outputas (
    vector< Studentas > & grupe)
```

7.17.1.11 sortByUser()

```
void sortByUser (
    vector< Studentas > & grupe,
    int temp)
```

7.17.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (
    const vector< Studentas > & grupe,
    vector< Studentas > & failed,
    vector< Studentas > & passed)
```

7.17.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

7.17.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

7.18 src/main_deque.cpp File Reference

```
#include "student_io_deque.h"
#include <deque>
```

Functions

- int [main](#) ()

7.18.1 Function Documentation

7.18.1.1 main()

```
int main ()
```

7.19 src/main_list.cpp File Reference

```
#include "student_io_list.h"  
#include <vector>
```

Functions

- int `main` ()

7.19.1 Function Documentation

7.19.1.1 `main()`

```
int main ()
```

7.20 src/main_vector.cpp File Reference

```
#include "student_io_vector.h"  
#include <vector>
```

Functions

- int `main` ()

7.20.1 Function Documentation

7.20.1.1 `main()`

```
int main ()
```

7.21 src/student.cpp File Reference

```
#include "student.h"  
#include <algorithm>  
#include <iostream>  
#include <utility>
```

Functions

- `std::istream & operator>>` (`std::istream &is`, `Studentas &s`)
Ivesties operatorius studento duomenims nuskaityti.
- `std::ostream & operator<<` (`std::ostream &os`, `const Studentas &s`)
Ivesties operatorius studento duomenims isvesti.

7.21.1 Function Documentation

7.21.1.1 `operator<<()`

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s)
```

Ivesties operatorius studento duomenims isvesti.

Parameters

<code>os</code>	Ivesties srautas.
<code>s</code>	<code>Studentas</code> objektas.

Returns

Ivesties srautas.

7.21.1.2 `operator>>()`

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s)
```

Ivesties operatorius studento duomenims nuskaityti.

Parameters

<code>is</code>	Ivesties srautas.
<code>s</code>	<code>Studentas</code> objektas.

Returns

Ivesties srautas.

7.22 src/utils.cpp File Reference

```
#include "utils.h"
#include <iostream>
#include <limits>
#include <cstdlib>
```

Functions

- void [intInput](#) (int &temp)
Nuskaityto sveikąjį skaičių ir patikrina investiciją.
- void [randomVardasPavarde](#) (string &vardas, string &pavarde)
Sugeneruoja atsitiktinį vardą ir pavardę.

Variables

- const vector< string > [vardai](#)
Vardų sąrašas atsitiktiniam generavimui.
- const vector< string > [vyr_pavardes](#) = {"Kazlauskas", "Jankauskas", "Petrauskas", "Stankevicius", "Zukauskas", "Butkus", "Pocius", "Urbonas", "Mockus", "Savickas"}
Vyrų pavardžių sąrašas atsitiktiniam generavimui.
- const vector< string > [mot_pavardes](#) = {"Kazlauskiene", "Jankauskiene", "Petrauskiene", "Stankeviciene", "Zukauskiene", "Butkiene", "Pociene", "Urboniene", "Mockiene", "Savickiene"}
Motėrių pavardžių sąrašas atsitiktiniam generavimui.

7.22.1 Function Documentation

7.22.1.1 intInput()

```
void intInput (
    int & temp)
```

Nuskaityto sveikąjį skaičių ir patikrina investiciją.

Parameters

<i>temp</i>	Kintamasis, į kurį įrašomas nuskaitytas skaičius.
-------------	---

7.22.1.2 randomVardasPavarde()

```
void randomVardasPavarde (
    string & vardas,
    string & pavarde)
```

Sugeneruoja atsitiktini vardą ir pavardę.

Parameters

<i>vardas</i>	Sugeneruotas vardas.
<i>pavarde</i>	Sugeneruota pavardė.

7.22.2 Variable Documentation

7.22.2.1 mot_pavardes

```
const vector<string> mot_pavardes = {"Kazlauskienė", "Jankauskienė", "Petrauskienė", "Stankevičienė",
    "Zukauskienė", "Butkienė", "Pocienė", "Urbonienė", "Mockienė", "Savickienė"}
```

Moteriskų pavardžių sąrašas atsitiktiniam generavimui.

7.22.2.2 vardai

```
const vector<string> vardai
```

Initial value:

```
= {"Jonas", "Mantas", "Tomas", "Lukas", "Karolis", "Darius", "Paulius", "Mindaugas", "Justas", "Rokas",
    "Agne", "Ieva", "Egle", "Gabija", "Monika", "Karolina", "Viktorija", "Emilija", "Justina",
    "Greta"}
```

Vardų sąrašas atsitiktiniam generavimui.

7.22.2.3 vyr_pavardes

```
const vector<string> vyr_pavardes = {"Kazlauskas", "Jankauskas", "Petrauskas", "Stankevičius",
    "Zukauskas", "Butkus", "Pocius", "Urbonas", "Mockus", "Savickas"}
```

Vyriskų pavardžių sąrašas atsitiktiniam generavimui.